Data Analysis + Regex + Python +

Understanding Regular Expressions (Regex)

With Real-World Examples

Joy Victor Follow 11 min read · Nov 2, 2023

167 10

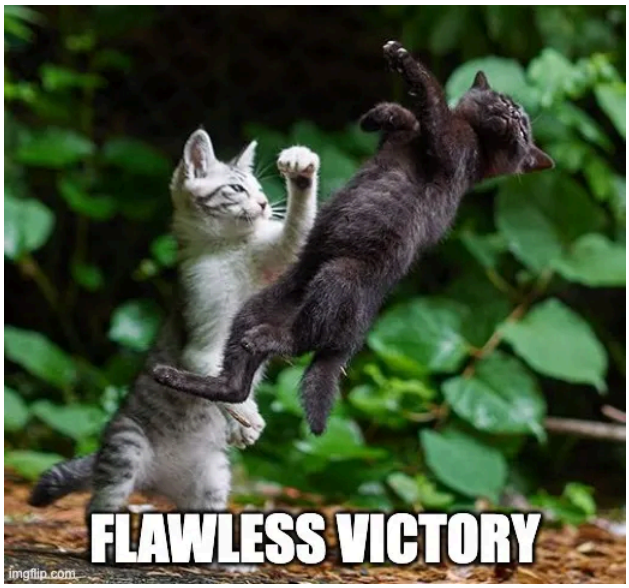


Credit: Search Engine Journal

During the just concluded DataFestAfrica, I spoke on Regular Expressions (Regex). Without a doubt, the conference was a huge success, with thousands of data practitioners gathering from all across Africa. My talk in particular made a huge impression on me. First of all, I had lots of people in my breakout session, which showed they all were on a quest to understand regular expressions. After the talk, I received tons of remarkable feedback on how my explanation made it extremely easy for them to understand a concept they had grappled with for so long. This influenced my decision to write an article anyone can refer to as the need arises.

Regex has always been an archnemesis. It has brought the most gallant of tech gurus to their knees, groveling to Stack Overflow and ChatGPT for help. At the end of this article, you'll have enough knowledge to put the grizzly beast, Regex to shame.

Medium Search Get app Write Sign up Sign in



Credit: imgflip.com

WHAT IS REGEX?

> *Regular Expression (Regex) is a sequence of characters that specifies a pattern in any given text (string).*

Regex can be used to search, extract, validate, or transform text based on specified patterns. This pattern can be as basic as searching for a comma (,) in a text or as complex as searching for a valid email address in a text.

WHY REGEX?

Regex comes in handy in Natural Language Processing (NLP), Web Scraping, Data Validation, Text Search and Manipulation, etc. Software Developers, Data Analysts, or Machine Learning Engineers need Regex at one point or the other. It's almost inevitable.

Fun Fact: Regex is implemented in various programming languages and text editors, making it a cross-disciplinary tool with a wide range of applications.

Before we proceed, please make sure you have pandas and regex packages installed and loaded in your local working environment. Alternatively, you can use this Google Colab notebook. It contains all the examples used in this article.

```
import pandas as pd
import regex as re
```

BASIC REGEX CHARACTERS

To successfully understand any concept, you must know the basics. Below are basic regex characters that build the foundation for Regex patterns. A regex character is simply a character(s) that has a special meaning in regular expressions.

- **Escape Character (backslash): **

This is used to begin most regex characters. It is also used when you want to interpret a character literally not as a special character. For example, full stop (.) means “Any character” in regex but if you want it to be interpreted literally, you use “\.”

Note: The code snippet below gives a better perspective into what each regex character does. Take a look!

```
test = "67+Joy$% an,.2"
result = re.findall("\.",test)

print(result)
```

```
['.']
```

test: A string with random letters, digits, special characters, and white space.

re.findall: A function in Python’s re module that matches a specific regex pattern and returns the result as a list. re.search, re.match, and re.sub are more functions in the re (regex) module. Read more here.

- **Any Character (full stop or dot):** .

This matches any character, whether it’s a digit, alphabet, or special character. See for yourself in the code snippet below.

```
test = "67+Joy$% an,.2"
result = re.findall(".",test)

print(result)
```

```
['6', '7', '+', 'J', 'o', 'y', '$', '%', ' ', 'a', 'n', ',', '.', '2']
```

- **Digit Character (backslash + lowercase letter d):** `\d`

This matches any digit 0–9.

```
test = "67+Joy$% an,.2"
result = re.findall("\d",test)

print(result)
```

```
['6', '7', '2']
```

- **Not a Digit Character (backslash + uppercase letter D):** `\D`

This is the opposite of the digit character (\d). It matches any character that is NOT a digit.

```
test = "67+Joy$% an,.2"
result = re.findall("\D",test)

print(result)
```

```
['+', 'J', 'o', 'y', '$', '%', ' ', 'a', 'n', ',', '.']
```

- **Word Character (backslash + lowercase letter w):** `\w`

This matches any character, from A-Z, 0–9, and underscore (_).

```
test = "67+Joy$% an,.2"
result = re.findall("\w",test)

print(result)
```

```
['6', '7', 'J', 'o', 'y', 'a', 'n', '2']
```

- **Not a Word Character (backslash + uppercase letter W):** `\W`

This is the opposite of the word character (\w). It matches any character that is NOT from A-Z, 0–9, and underscore (_).

```
test = "67+Joy$% an,.2"
result = re.findall("\W",test)

print(result)
```

```
['+', '$', '%', ' ', ',', '.']
```

- **Whitespace Character (backslash + lowercase letter s):** `\s`

This matches any whitespace like space, tab, or a newline.

```
test = "67+Joy$% an,.2"
result = re.findall("\s",test)

print(result)
```

```
[' ']
```

- **Not a Whitespace Character (backslash + uppercase letter S):** `\S`

This is the opposite of the space character (\s).

```
test = "67+Joy$% an,.2"
result = re.findall("\S",test)

print(result)
```

```
['6', '7', '+', 'J', 'o', 'y', '$', '%', 'a', 'n', ',', '.', '2']
```

- **Beginning of a String Character (caret):** `^`

This character matches the beginning of a string. It is useful for matching strings that start with a specific pattern. For example; The pattern below means select the string that starts with number 6.

```
test = "67+Joy$% an,.2"
result = re.findall("^6",test)

print(result)
```

```
['6']
```

- **End of a string(dollar):** `$`

This character matches the end of a string. It is useful for matching strings that end with a specific pattern. For example; The pattern below means to select the string that ends with a number 2.

```
test = "67+Joy$% an,.2"
result = re.findall("2$",test)

print(result)
```

```
['2']
```

GROUPINGS

Sometimes you want a regex pattern to match a group of characters that are arranged a certain way, not just random digits or alphabets. For example, if you want to collect phone numbers that are typed this way, +123456–7890 then you need groupings.

- **Character Class (Square brackets):** `[ ]`

This is used to define a character class. A character class is a set of characters that can all be matched by the regex. For example, the character class [aeiou] matches any vowel.

- **Negated Character (Square brackets with caret):** `[^ ]`

This is the opposite of the character class. It matches any character that is NOT in the specified set. For example, [^aeiou] matches any character that is not a vowel.

- **Alternation (Pipe):** `|`

Also known as Either Or, this behaves like an OR operator. It either matches the expression on the left or the one on the right. For example, Miss|Mr matches either Miss or Mr.

- **Capturing group (Parenthesis):** `( )`

This is used to match a group of characters that can be extracted later. For example, (ab) matches the characters ab only.

QUANTIFIERS

Quantifiers in regex are used to specify the number of times a character or group of characters should be matched. They can be added with other regex characters to create more complex regex patterns.

- **0 or more (Asterix):** `*`

This means the preceding regex character or group of characters should match zero or more times. For example, \d* means zero or more digits.

- **1 or more (Plus):** `+`

This means the preceding regex character or group of characters should match one or more times. For example, \d+ means one or more digits.

- **0 or 1 (Question Mark):** `?`

This means the preceding regex character or group of characters should match zero or one time. For example, \d? means zero or one digit.

- **An exact number of characters (Curly brackets):** `{ }`

This means the regex character or group of characters in the curly bracket should match the specified number of times. For example, a{3} matches “aaa” but not “aa” or “a.”

- **Range of number of characters(Curly brackets with range):** `{Minimum, Maximum}`

This means the regex character or group of characters in the curly bracket should match the specified range of values. For example, a{2,4} matches “aa,” “aaa,” and “aaaa,” but not “a” or “aaaaa.”

REAL-WORLD EXAMPLES

If you have come this far, well done! You're gradually on your way to taming the beast. As earlier mentioned, regex applies to different forms of data. For the sake of this article, we will be working with dummy survey data in a pandas dataframe. Survey data is notorious for being very dirty, primarily because people have different levels of understanding and ways of expression. This dummy data mimics typical survey data where regex can be applied. The data would be created and stored in a dataframe using the pandas library from Python.

Get Joy Victor's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

We will focus on three examples that contain numbers, letters, and special characters. This will reinforce your understanding of regular expressions and allow you to practice. If you haven't done this already, please make sure you have pandas and regex packages installed and loaded in your local working environment. Alternatively, you can use this Google Colab notebook that contains all the examples.

Example 1

The dataframe (as shown below) is a list of respondents with their titles, first names, and surnames. Please follow the code snippet below to create your dataframe.

```
# Create a dataframe with names
data = ["Miss Joy Victor", "Sis Grace Lakowe", "Mr Adekunle Bisong", "Dr Peace Chima", "Alhaji Adamu Rano"]
df = pd.DataFrame(data, columns=['Full_Name'])
df
```

	Full_Name
0	Miss Joy Victor
1	Sis Grace Lakowe
2	Mr Adekunle Bisong
3	Dr Peace Chima
4	Alhaji Adamu Rano

The task is to split this single information (Full_Name) into different columns (Title, First_Name, Surname), each containing their respective title, first name, and surname. This makes it easier to analyze the respondent's data. The key to creating a regex pattern is observing the similarities in the data. Take a closer look at the dataframe above again, you'll notice that the title, first name, and surname are separated by the space character. All you have to do now is to define a regex pattern to separate the Full_Name column into a new column whenever it encounters a space character.

```
# Regex to split the columns
pattern = "(Mr|Ms|Miss|Mrs|Dr|Alhaji|Sis)\s+(\w+)\s+(\w+)"
df[["Title", "First_Name", "Surname"]] = df["Full_Name"].str.extract(pattern)
df
```

	Full_Name	Title	First_Name	Surname
0	Miss Joy Victor	Miss	Joy	Victor
1	Sis Grace Lakowe	Sis	Grace	Lakowe
2	Mr Adekunle Bisong	Mr	Adekunle	Bisong
3	Dr Peace Chima	Dr	Peace	Chima
4	Alhaji Adamu Rano	Alhaji	Adamu	Rano

pattern: The regex pattern you have in mind to match

df[[“Title”, “First_Name”, “Surname”]]: This creates new columns with the headers; Title, First_Name, and Surname respectively.

df[“Full_Name”].str.extract(pattern): The str.extract() function in Python is used to extract capture groups that match the specified regex pattern and separate them as columns in a dataframe.

And just like that, we have three new columns containing the respondent’s title, first name, and surname. Now let’s interpret the regex pattern step by step, going from left to right.

```
(Mr|Ms|Miss|Mrs|Dr|Alhaji|Sis)\s+(\w+)\s+(\w+)
```

(Mr|Ms|Miss|Mrs|Dr|Alhaji|Sis): All titles. This is the **first capture group** to match titles in the Full_Name column, i.e Mr or Ms or Miss or Mrs etc.

\s+: To account for one or more space characters.

(\w+): All First names. This is the **second capture group** to match all word characters after the first space character(s).

\s+: To account for one or more space characters.

(\w+): All Surnames. This is the **third capture group** to match all word characters after the second space character(s).

Alternatively

The cool thing about regex is its flexibility. You can create different patterns and achieve the same result. Using the first example above, here’s an alternative regex pattern to achieve the same result.

```
pattern = "([A-Z][a-z]+)\s+([A-Z][a-z]+)\s+([A-Z][a-z]+)"
# [A-Z][a-z] means all alphabets
df[["Title", "First_Name", "Surname"]] = df["Full_Name"].str.extract(pattern)
df
```

	Full_Name	Title	First_Name	Surname
0	Miss Joy Victor	Miss	Joy	Victor
1	Sis Grace Lakowe	Sis	Grace	Lakowe
2	Mr Adekunle Bisong	Mr	Adekunle	Bisong
3	Dr Peace Chima	Dr	Peace	Chima
4	Alhaji Adamu Rano	Alhaji	Adamu	Rano

```
([A-Z][a-z]+)\s+([A-Z][a-z]+)\s+([A-Z][a-z]+)
```

([A-Z][a-z]+): All titles. This is the **first capture group** to match all uppercase letters from A to Z, followed by lowercase letters from a to z in the Full_Name column.

\s+: To account for one or more space characters.

([A-Z][a-z]+): All First names. This is the **second capture group** to match all uppercase letters from A to Z, followed by lowercase letters from a to z.

\s+: To account for one or more space characters.

([A-Z][a-z]+): All Surnames. This is the **third capture group** to match all uppercase letters from A to Z, followed by lowercase letters from a to z.

Example 2

The dataframe (as shown below) is a list of respondent's phone numbers. Please follow the code snippet below to replicate.

```
number = ["+234-9087221478", "+72-89023432", "+92-89652893", "+011-1342345", "+5-982400700012"]
number_df = pd.DataFrame(number, columns=['Phone_Number'])
number_df
```

	Phone_Number
0	+234-9087221478
1	+72-89023432
2	+92-89652893
3	+011-1342345
4	+5-982400700012

The task is to split the Phone Numbers into Area Codes and Line Numbers. Remember, the key to creating a regex pattern is observing the similarities in the data. Take a closer look at the dataframe above again, you'll notice that the Area Code starts with a plus(+) sign while the Line Number is preceded by a minus sign or hyphen (-). All you have to do now is to define a regex pattern to separate the Area_Code from the Line_Number whenever it encounters the minus sign or hyphen (-).

```
pattern = "(\+\d+)\-(\d+)"
number_df[["Area_Code", "Line_Number"]] = number_df["Phone_Number"].str.extract(pattern)
number_df
```

	Phone_Number	Area_Code	Line_Number
0	+234-9087221478	+234	9087221478
1	+72-89023432	+72	89023432
2	+92-89652893	+92	89652893
3	+011-1342345	+011	1342345
4	+5-982400700012	+5	982400700012

Well, that was easy. Now let's interpret the regex pattern step by step, going from left to right.

```
(\+\d+)\-(\d+)
```

(\+\d+): Area Code. This is the **first capture group** to match all digits before the hyphen (-) including the plus sign (+)

\-: To account for the hyphen(-)

(\d+): Line Number. This is the **second capture group** to match all digits after the hyphen (-)

Alternatively

Speaking of regex's flexibility, here's an alternative regex pattern to achieve the same result.

```
# Alternatively
pattern = "(\+[0-9]+)\-([0-9]+)"
# [0-9] means all positive integers
number_df[["Area_Code", "Line_Number"]] = number_df["Phone_Number"].str.extract(pattern)
number_df
```

	Phone_Number	Area_Code	Line_Number
0	+234-9087221478	+234	9087221478
1	+72-89023432	+72	89023432
2	+92-89652893	+92	89652893
3	+011-1342345	+011	1342345
4	+5-982400700012	+5	982400700012

```
(\+[0-9]+)\-([0-9]+)
```

(\+[0–9]+): Area Code. This is the **first capture group** to match any digit from 0–9 before the hyphen (-) including the plus sign (+)

\-: To account for the hyphen (-)

(\d+): Line Number. This is the **second capture group** to match any digit from 0–9 after the hyphen (-)

Example 3

The dataframe (as shown below) is a list of respondent's house addresses. Please follow the code snippet below to replicate.

```
# Create a dataframe with addresses
address = ["89 Alapere, Indiana", "2A Aba, Oyibo", "AA Badagri, Chinatown", "71 Oregun, Lagbaja", "912 Bloomingdale, Freetown"]
address_df = pd.DataFrame(address, columns=["Address"])
address_df
```

	Address
0	89 Alapere, Indiana
1	2A Aba, Oyibo
2	AA Badagri, Chinatown
3	71 Oregun, Lagbaja
4	912 Bloomingdale, Freetown

The task is to split the Addresses into House Numbers, Streets, and Cities. By now, you should already be familiar with the process. Observe the similarities and create a regex pattern to suit. The similarity in this case is; the House_Number and Street name are separated by a space character while the Street name and City are separated by a comma (,) and a space character.

```
pattern = "(\w+)\s+(\w+)\,\s+(\w+)"
address_df[["House_Number", "Street", "City"]] = address_df["Address"].str.extract(pattern)
address_df
```

	Address	House_Number	Street	City
0	89 Alapere, Indiana	89	Alapere	Indiana
1	2A Aba, Oyibo	2A	Aba	Oyibo
2	AA Badagri, Chinatown	AA	Badagri	Chinatown
3	71 Oregun, Lagbaja	71	Oregun	Lagbaja
4	912 Bloomingdale, Freetown	912	Bloomingdale	Freetown

Without further ado, let's interpret this step by step, going from left to right.

```
(\w+)\s+(\w+)\,\s+(\w+)
```

(\w+): All House_Number: This is the **first capture group** to match all word characters.

\s+: To account for one or more space characters.

(\w+): All Street: This is the **second capture group** to match all word characters after the first space character(s).

\,: To account for the comma after the name of the street.

\s+: To account for one or more space characters.

(\w+): All City: This is the **third capture group** to match all word characters after the second space character(s).

CONCLUSION

With these examples, I hope you can finally say you have enough knowledge to put the grizzly beast, Regex to shame. After this article, I encourage you to keep practicing so it becomes second nature to you. If you wish to continue

your quest for knowledge, I have added some resources below for you to check out. As always,

Stay data savvy,
Joy (Jhoie) Victor 💕

RESOURCES

1. https://towardsdatascience.com/regular-expressions-clearly-explained-with-examples-822d76b037b4
2. https://www.youtube.com/watch?v=r6I-Ahc0HB4&list=PL4cUxeGkcC9g6m_6Sld9Q4jzqdqHd2HiD
3. https://regex101.com/

Data Analysis + Regex + Python +



Written by Joy Victor

259 followers · 64 following

Follow

Data Analyst 🚀 Sitting pretty at the intersection of data, ethics, and research

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech